

Cross-Wallet PID Issuance via App Clip

How any wallet obtains a formally-verified Person Identification Datum (PID) by launching the standalone **PID Capture** companion (a full app or an install-free App Clip) that reads an eMRTD passport chip over NFC and confirms a live human with iProov Genuine Presence Assurance — with the credential minted straight back into the wallet that started the flow, bound to that wallet's own key.

Scope: the shipped capture-session backend, the companion app/App Clip, and the cross-wallet correlation that routes the issued PID to the requesting wallet. · **Issuer:** `issuer.advatar.systems` · **Capture host:** `pid.advatar.systems` · **Credential:** German PID, SD-JWT VC (`dc+sd-jwt`).

1 · At a glance

PID Capture decouples **who proves the identity** from **who holds the credential**. A wallet that cannot itself read a passport chip — a thin wallet, an App-Clip-only experience, or a wallet on another device — opens a short-lived **capture session** bound to its own proof-of-possession key. The companion performs the sensitive capture (chip, then liveness + face match), the issuer validates it authoritatively, and the resulting PID is bound to the requesting wallet's key. The companion never holds the credential and never sees a signing key.

1. Bind

The wallet generates a P-256 holder key and opens a session; the PID will be bound to that key's `cnf` .

2. Capture

A universal link / App Clip launches PID Capture: a chip read over the `service-nfc` relay, then an iProov GPA scan that proves liveness and matches the face to the chip portrait.

3. Issue

VCIssuer re-runs Passive Authentication and validates the liveness + portrait-match verdict, then mints the PID and hands it back to the wallet.

Components

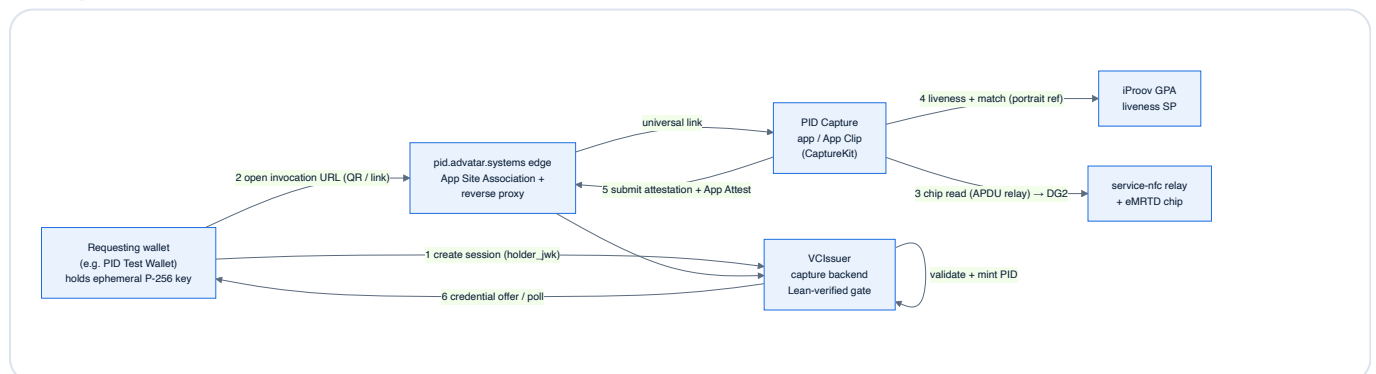


Figure 1. Component map. The `pid.advatar.systems` edge serves the Apple App Site Association (so the link opens the app) and proxies the capture API to the issuer.

2 · End-to-end sequence

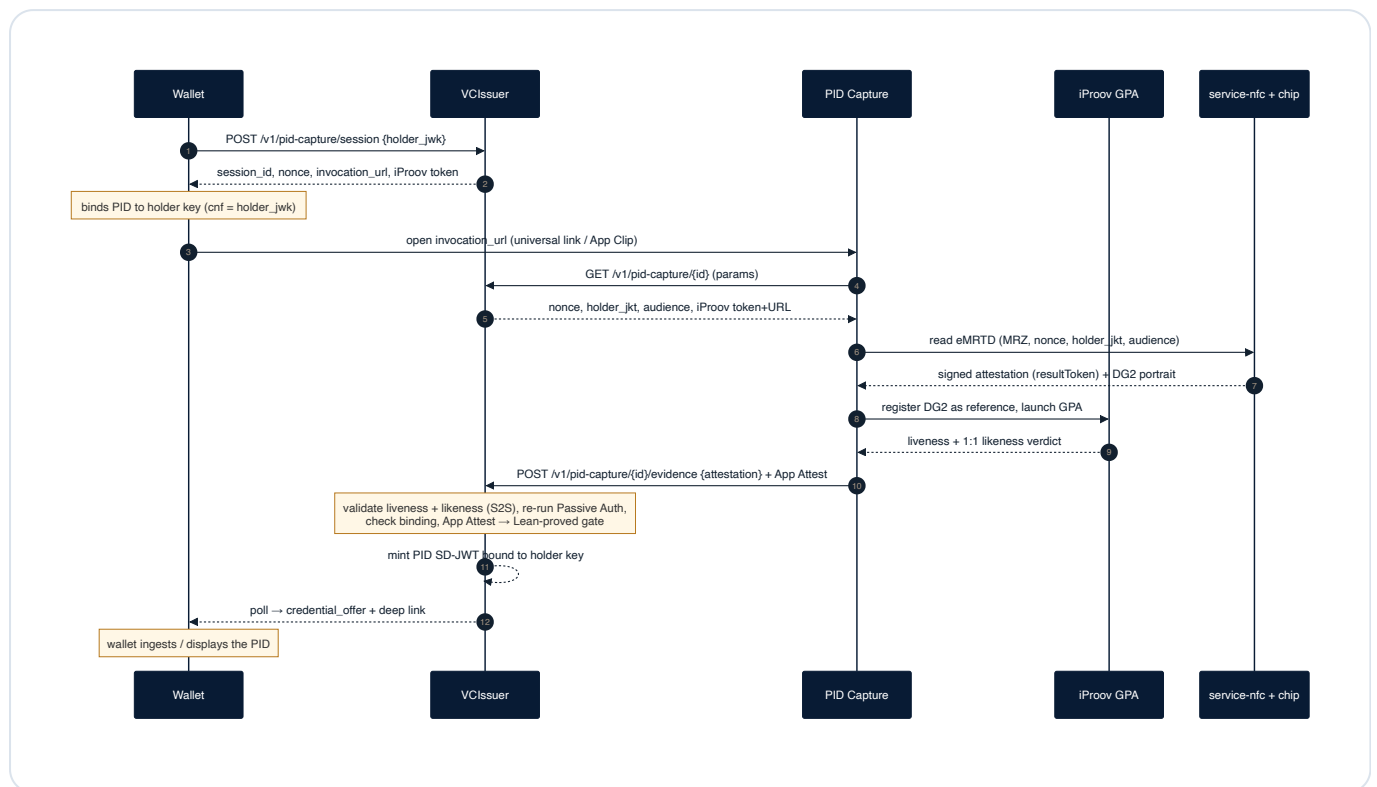


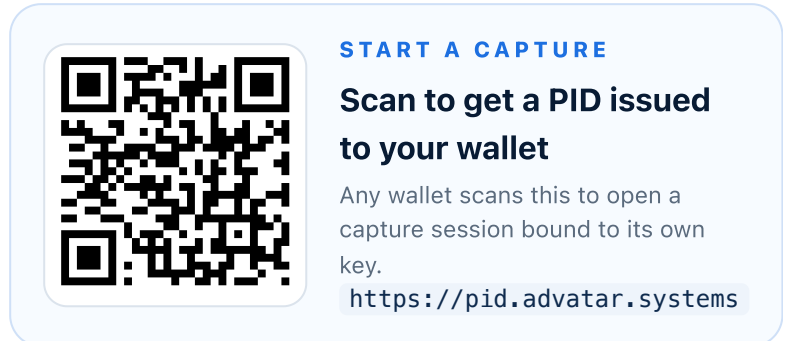
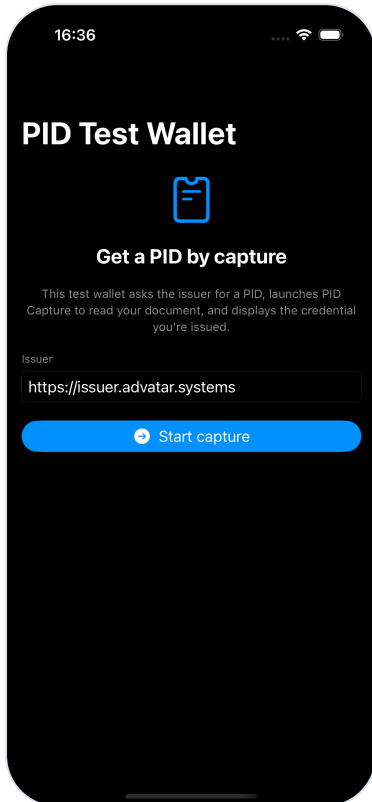
Figure 2. The full capture-to-credential sequence, in the correct order (chip → GPA so the DG2 portrait is the match reference). The `/v1/pid-capture` handlers are as shipped; the current `CaptureCoordinator.run` still runs liveness before the chip, so reordering it to this is a tracked follow-up.

Why this shape? The wallet registers its key *first*, so the PID is bound to the wallet by construction — there is no redirect chain carrying a bearer credential. The companion only ever produces *evidence*; the issuer makes the authoritative decision and is the only party that can sign.

3 · Walkthrough with screens

■ Live screenshot ■ Representative UI mockup (device-only screen)

Step 1 — Start from any wallet LIVE

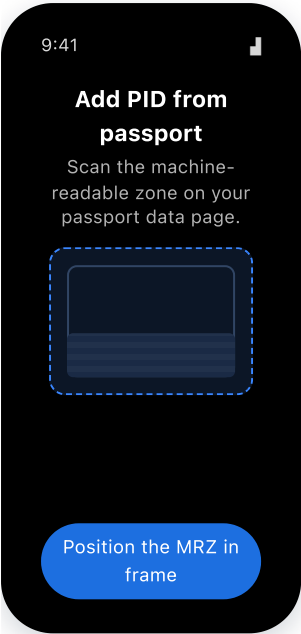


VC Issuer landing QR. Reproduced from the WebIssuer *PID Capture* section; the code encodes `https://pid.advatar.systems`.

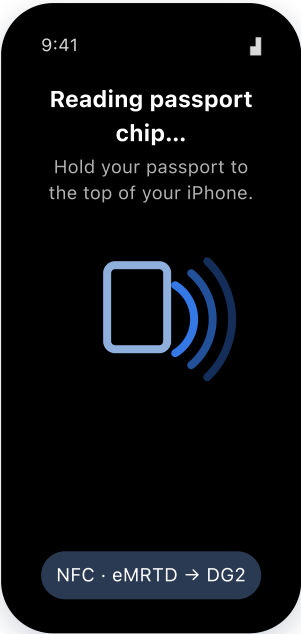
PID Test Wallet. A minimal reference wallet: it creates a key-bound session, launches PID Capture, and displays the PID it receives. It stores nothing.

Step 2–6 — Inside PID Capture

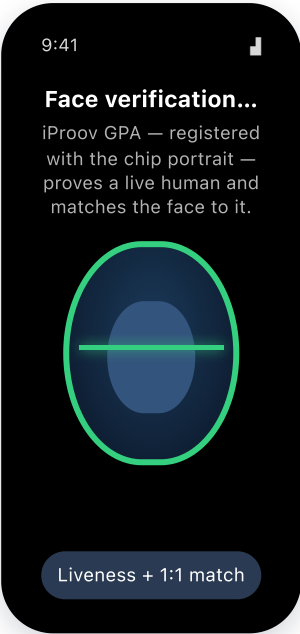
REPRESENTATIVE MOCKUPS



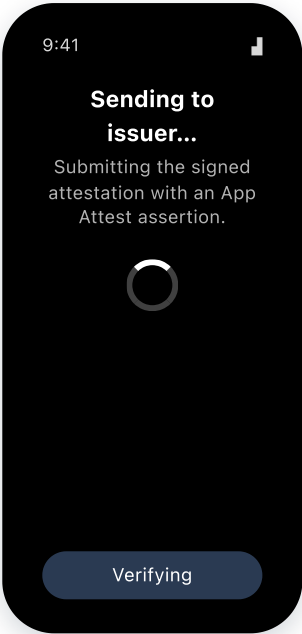
MRZ scan
camera reads the data page



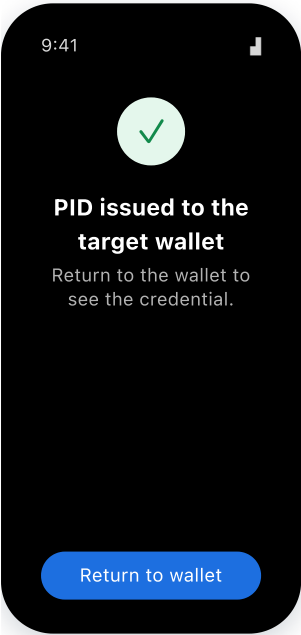
NFC chip read
portrait (DG2) + authenticity



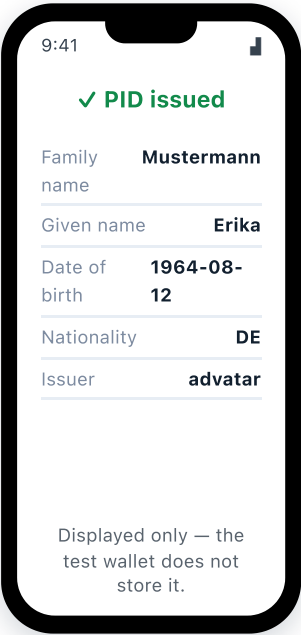
iProov GPA
liveness + likeness



Submit
evidence to VCIssuer



Issued
hand back via deep link



Wallet displays PID
bound to its key

Note on the device screens. PID Capture runs only on a physical iPhone — iProov’s liveness SDK and CoreNFC are unavailable in the iOS Simulator — so the six screens above are faithful **representative mockups** built from the app’s exact on-screen copy. Only the home screen (Step 1, left) and the QR are live output.

4 · Backend NFC — reading the passport chip

The passport's contactless chip is an **eMRTD** (ICAO 9303). PID Capture uses the `ChipmunkNFC` SDK, which drives CoreNFC on the phone but runs the eMRTD protocol through the **service-nfc WebSocket relay** (`wss://nfc.dev-eu.iproov.id/channel`). The phone is the radio bridge; the cryptographic session and the document's authenticity checks are handled server-side and returned as a **signed attestation**.

What the read includes

- **Access** — BAC/PACE derived from the MRZ (passport number, DOB, expiry).
- **DG2 portrait** — the chip's facial image (`skipDG2: false`).
- **Passive Authentication** (`doPA`) — verifies the Document Security Object signature chains to a CSCA.
- **Active Authentication** (`doAA`) — challenge–response proving the chip is genuine, not cloned.

The binding

The relay welds the VCIssuer session context — **session nonce**, **target holder-key thumbprint**, and **audience** — into the signed `resultToken` it returns. That ties this specific chip read to this specific capture session and this specific wallet, so evidence cannot be replayed or re-pointed at another wallet.

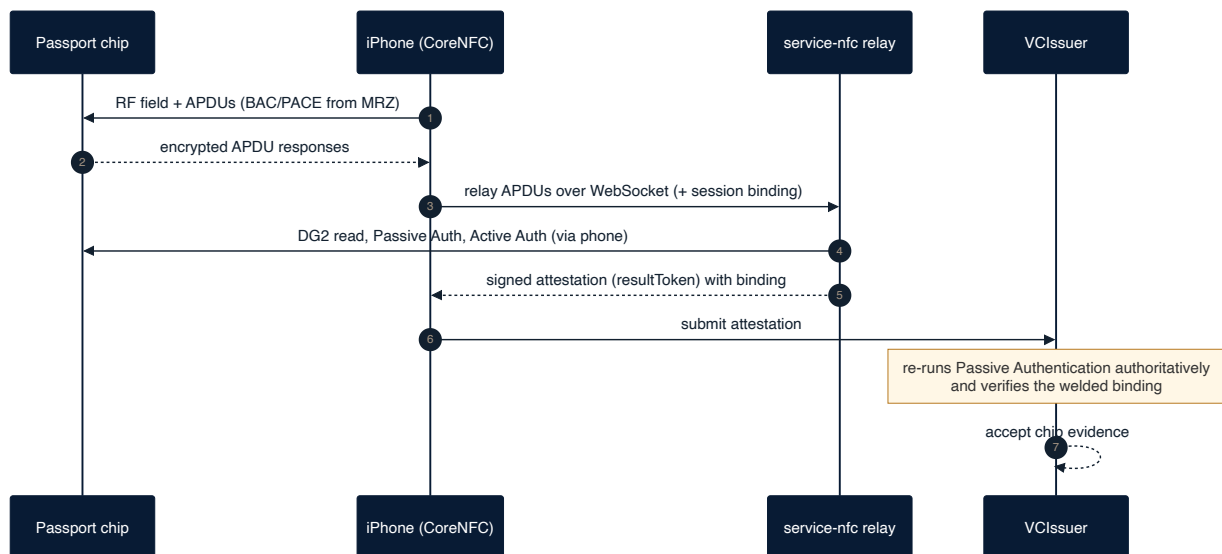


Figure 3. Backend NFC. The phone bridges RF; the relay drives the eMRTD session and signs the result; VCIssuer re-verifies Passive Authentication itself — the client is never trusted to decide.

5 · iProov — liveness and likeness

Two distinct checks are involved, and separating them explains the order. **Liveness** (iProov Genuine Presence Assurance) asks *“is this a real, present human?”* — a brief, screen-illuminated face scan that resists photos, masks and replays. **Liikeness** (a 1:1 face match) asks *“is that live face the same person as the passport photo?”* A likeness check needs a reference photo, and the only trustworthy one is the chip’s DG2 portrait — so the chip is read *first*, its portrait is registered with iProov as the reference, and the GPA scan runs *after*, settling liveness and likeness in one capture. VCIssuer mints the iProov token **server-to-server** when the session opens, so the client never chooses its own parameters.

Authoritative at the issuer

The SDK reports only whether the capture *completed*. The authoritative pass/fail is decided by VCIssuer validating the result against iProov’s API — **downgrade-closed**: if the liveness provider is unconfigured or unreachable, issuance fails rather than proceeding without proof.

Why the chip is read first

The likeness match needs a reference face, and the authoritative one is the chip’s DG2 portrait — which only exists after the NFC read. So the sequence is **MRZ → NFC (portrait) → GPA**: iProov is registered with that portrait and the scan proves a live human *and* matches it. Running GPA before the chip could only ever prove liveness, never likeness.

How the match happens. The chip’s DG2 portrait is registered with iProov as the reference; the GPA capture then returns a combined **liveness + 1:1 likeness** verdict. VCIssuer re-validates that verdict server-to-server and requires it alongside verified Passive/Active Authentication — the Lean-proved gate refuses to mint without **portrait-matched liveness**. Together they answer the issuer’s two questions: **is the document genuine**, and **is the live person its subject**.

6 · Security properties

Property	How it is enforced
Holder-key binding	The PID's <code>cnf</code> is the wallet's P-256 JWK, registered at session creation. Only that wallet's key can later present the credential.
Session binding & anti-replay	A per-session nonce and the target holder-key thumbprint + audience are welded into the eMRTD attestation and checked by the issuer.
Genuine companion	The evidence submission carries an App Attest assertion bound to the request body, so VCissuer can require a real, registered PID Capture instance.
Authoritative validation	The iProov liveness + likeness verdict (S2S) and Passive Authentication are re-validated at the issuer , downgrade-closed. The client is never trusted to decide.
Short-lived sessions	A capture session accepts evidence for 900 s , then must be recreated.
Verified issuance gate	The signing decision is a pure Rust kernel mirrored by a Lean 4 model and Tamarin lemmas; only the kernel can authorize a signature. An optional hybrid ES256 + ML-DSA-65 (post-quantum) wrapper is available.

7 · App Clip & universal-link mechanics

PID Capture claims `applinks:pid.advatar.systems` (and `appclips:pid.advatar.systems`). The `pid.advatar.systems` edge serves the Apple App Site Association at `/.well-known/apple-app-site-association` for app id `L2AF8KFX35.eu.advatar.wallet.pidcapture` (and `.Clip`), so opening `https://pid.advatar.systems/pid-capture?session=...` launches the installed app — or an **App Clip** with no App Store install. The same host reverse-proxies the capture API (`/v1/pid-capture/*`) to the issuer, so the companion resolves everything from one origin.

App
Full PID Capture install; universal link opens it directly.

App Clip
Instant, install-free; links only
CaptureKit to stay under Apple's
15 MB budget.

Cross-device
A wallet on another device polls
the session and receives the PID
by credential offer.

Advatar EUDI Wallet — Cross-Wallet PID Issuance via App Clip. Endpoints: `POST /v1/pid-capture/session`, `GET /v1/pid-capture/{id}`, `POST /v1/pid-capture/{id}/evidence`. Credential configuration `eu.europa.ec.eudi.pid_vc_sd_jwt.de` · format `dc+sd-jwt`. Device screens are representative mockups; home screen and QR are live output.